

Foxtrot Overview

Foxtrot optimizes the sequence of a route in real time. A route is considered to be a set of stops (waypoints), which usually take time (service time), have a set of tasks to complete (deliveries), and optionally have time windows.

Foxtrot not only optimizes a route throughout its operation, but it also learns about your customers' time windows, locations and service times from the drivers' behavior. While these parameters are optional, providing them will help Foxtrot yield better optimization results.

Distribution Centers

For most API calls, a Distribution Center (see [DC](#)) ID must be specified. A list of all the DCs for an organization can be obtained by calling [GET DCs](#).

Drivers

Create, read, update, and delete individual drivers (see [Driver](#)) or all drivers for a single distribution center can be obtained using the [Drivers API](#).

Driver authentication is done at the SDK level using the `login_token`. See the [SDK documentation](#) for more details.

The Route Lifecycle

A **Route** typically is imported before the driver starts their day. A route is composed of its unique Route ID, an estimated start date, waypoints, and more. Details can be found in [the create route API](#).

Note

If an incorrect route is accidentally imported or it needs to be modified, that route must first be **DELETED** and then import it again.

Typically, using a Foxtrot's mobile SDK integrated application, drivers will obtain their corresponding **active** route based on their credentials and driver ID. To create a route, please refer to: [Create Route](#). To [activate](#) or [deactivate](#) an individual route after importing, use the route's ID.

After a driver completes all waypoints in a route and finishes it, Foxtrot's mobile SDK will look for the next (based on planned route **start time**) **active** unfinished route [assigned](#) to them.

API Overview

Foxtrot's API can be accessed using the following URL format:

```
https://apiv1.foxtrotsystems.com/{ENDPOINT}
```

Authentication

Foxtrot's API works with the API key provided during account setup. The API key has many privileges to access **all data** in the organization, so make sure it is kept secret! **Do not** share or store the API key online or expose it to the internet (places such as client-side code, GitHub, etc). Please keep this key in a secure location and avoid sending it over the internet unless connecting to Foxtrot's servers over **HTTPS**.

Here's an example call that will return distribution centers for an organization:

```
curl https://apiv1.foxtrotsystems.com/dcs \
-X GET \
-H "Content-Type: application/json" \
-H "Authorization: Bearer {{YOUR_API_KEY}}"
```

All API calls must include the *Authorization* header in order to access any information.

Request Structure

All API requests must have the following header values:

- `Content-Type` must be set to `application/json`.
- `Authorization` must be set to `Bearer {{YOUR_API_KEY}}`.

Response Structure

All API responses are formatted as follows:

```
{
  "status": "the status",
  "data": {
    "sample_key": "sample_value",
    "sample_key2": "sample_value2"
  }
}
```

The `status` key will contain one of these values:

- `success` - The request has been successfully processed.
- `pending` - The request has been accepted, but is still being processed.
- `error` - An error occurred while processing the request. Please consult the individual endpoint documentation for more details. Both the HTTP status code the and `data` key will reveal more about what went wrong.

The `data` key contains relevant information about the request. Please consult the individual API endpoint documentation for more detail.

Sample success response

```
{
  "status": "success",
  "data": {
    "sample_key": "sample_value"
  }
}
```

Sample pending response

```
{
  "status": "pending",
  "data": {
    "sample_key": "sample_value"
  }
}
```

Sample error response

```
{
  "status": "error",
  "data": {
    "code": 1000,
    "message": "error message",
    "sample_key": "sample_value"
  }
}
```

The Foxtrot API doesn't always return a `code` field within the error response, but if it does, please consult the [Error Codes](#) table to implement appropriate error handling for that error code. If the response does not have the code, please log the response. The API will try to return a useful error message for debugging.

Error Codes

Error Code	Description
400	Bad parameters. Please check the documentation of the API to make sure you pass in all requi
1000	Please contact us at support@foxtrot.io .
1001	User was not fully configured.

Error Code	Description
1002	DC was not fully configured.
2000	Driver with that id already exists.
2001	Driver with that id does not exist.
3000	There was an error with the supplied route. Please check the <code>data.errors</code> key for more details
3001	The time windows are too restrictive. Please extend the window and try again.
3002	The stop cannot be at a warehouse.
3003	Route with that id does not exist.
3004	Cannot delete already analyzed routes.
3005	Cannot import another route with the same id.
3006	Cannot import another waypoint with the same id.
3007	Cannot import another delivery with the same id.
3008	Cannot import another invoice with the same id.
3009	Cannot import another invoice item with the same id.
5001	Route accounting record with that id already exists.

Basic Models

These are some of the common models that will be returned by some of the API. Check the endpoint’s documentation if it returns a different variant of the model.

DC

Describes a DC

```
{
  "id": "the dc's id",
  "name": "the dc's name",
  "time_zone": "America/Los_Angeles"
}
```

Property	Type	Nullable	Description
id	string	false	The id of the dc.
name	string	false	The name of the dc.
time_zone	string	false	The time zone the dc is in.

Driver

Describes a Driver

```
{
  "id": "the driver's id",
  "name": "the driver's name",
  "login_token": "abcde-fghikj"
}
```

Property	Type	Nullable	Description
id	string	false	The id of the driver.
name	string	false	The name of the driver.
login_token	string	false	The token used to login through Foxtrot's mobile SDK

Note

The login token is used to login this driver to the SDK. These can be pre-fetched, stored and sent to the mobile app implementing the SDK after successful authentication.

Route

Describes a Route

```
{
  "id": "route id",
  "name": "route name",
  "assigned_driver_id": "driver id",
  "start_time": 1510622168000,
  "start_warehouse_id": "the warehouse id",
  "end_warehouse_id": "the warehouse id",
  "vehicle_id": "vehicle id",
  "version": 1,
  "waypoint_ids": [
    "waypoint id"
  ],
  "is_active": false,
  "is_finalized": false,
}
```

Property	Type	Nullable	Description
id	string	false	The id of the route.
name	string	false	The name of the route.
assigned_driver_id	string	true	The id of the driver assigned to this route.
start_time	long	false	The start time of the route as a UNIX epoch since Janu
start_warehouse_id	string	false	The id of the starting warehouse.
end_warehouse_id	string	false	The id of the destination warehouse.
vehicle_id	string	true	The id of the vehicle associated to this route.
version	integer	false	The version of the route. This value changes whenever
waypoint_ids	Array[string]	false	An array of waypoint ids representing the route sequer
is_active	boolean	false	This value control whether or not a driver can see the r
is_finalized	boolean	false	This value control whether or not a driver has complete

Delivery Status

An enumeration of possible delivery statuses.

Method	Description
<i>FAILED</i>	(At Delivery Site: Option 1/3) Marks a failed delivery.
<i>SUCCESSFUL</i>	(At Delivery Site: Option 2/3) Marks a successful delivery.
<i>VISIT_LATER</i>	(At Delivery Site: Option 3/3) Marks the delivery to be visited later.
<i>AUTHORIZE_REATTEMPT</i>	(Optional: After prior DeliveryAttempt) Marks the delivery to be reattempted la

Route Completion Time

Status of a route completion time.

Property	Type	Nullable	Description
type	string	false	Can be ESTIMATED, ACTUAL, or FINALIZED
timestamp	long	false	Completed timestamp as a UNIX epoch since January 1st, 1970 in mil

Route Completion Time Type

An enumeration of possible delivery statuses.

Method	Description
<i>ESTIMATED</i>	Route is still in progress so completion time is an estimate.
<i>FINALIZED</i>	Driver finished the route, but route is pending analysis.
<i>ACTUAL</i>	Route is completed.

Importing Routes

Importing a route requires using our asynchronous [import route](#) API. The API will check for possible errors in the input and then return a transaction id. This transaction id should be used to [check for the import status](#) of that route. A route is not successfully imported until the import status API returns a successful response. Errors are returned as soon as possible, but network latency may prevent all error detection from happening at the time of route import (e.g. duplicate request to send the same route) so it is possible for error to come back from the import status API.

The following is a sample pseudocode to import a route.

Important

In order for a driver to receive a route, it must be [activated](#) after a successful import or you can set the *activate_after_successful_import* flag to true during route creation.

supplied, Foxtrot will use the default warehouse location for that DC (Under Account > Settings > DC > Warehouse).

❗ Important

Route id has to be globally unique. You cannot have multiple routes with the same id.

❗ Important

If you are not sure the driver has been imported into Foxtrot, pass in the *driver* parameter and the driver will be imported if the driver does not exist. *driver_id* and *driver* cannot be used together.

Property	Type	Required	Description
id	string	true	The globally unique id of the route.
name	string	false	The name of the route, can be the same as the driver name.
driver_id	string	false	The id of the driver to be assigned to this route.
driver	Driver	false	The driver assigned to this route. If the driver does not exist, it will be imported.
activate_after_successful_import	boolean	false	Setting to automatically activate a route after a successful import.
start_time	long	true	The start time of the route as a UNIX timestamp.
warehouse	Warehouse	false	The route start and end location. If the warehouse is not specified, the default warehouse for the DC will be used.
start_warehouse	Warehouse	false	The route starting location. <i>end_warehouse</i> must also be specified.
end_warehouse	Warehouse	false	The route ending location. <i>start_warehouse</i> must also be specified.
waypoints	Array[Waypoint]	true	An array of waypoint objects.

Property	Type	Required	Description
vehicle	Vehicle	false	The vehicle to be associated to this r



```

{
  "route": {
    "id": "globally unique route id",
    "name": "route name",
    "driver_id": "globally unique driver id",
    "activate_after_successful_import": false,
    "start_time": 1510860915000,
    "start_warehouse": {
      "id": "globally unique warehouse id",
      "name": "warehouse name",
      "address": "physical address",
      "location": {
        "latitude": 3.0,
        "longitude": 4.0
      }
    },
    "end_warehouse": {
      "id": "globally unique warehouse id",
      "name": "warehouse name",
      "address": "physical address",
      "location": {
        "latitude": 3.0,
        "longitude": 4.0
      }
    },
    "waypoints": [{
      "id": "globally unique waypoint id",
      "customer_id": "customer id",
      "name": "waypoint name",
      "address": "physical address",
      "location": {
        "latitude": 1.0,
        "longitude": 2.0
      },
      "service_time_minutes": 999,
      "time_windows": [{
        "start": 1510864915000,
        "end": 1510868515000
      }],
      "deliveries": [{
        "id": "globally unique delivery id",
        "product": "product sku or description",
        "quantity": 4355.0
      }]
    }],
    {
      "id": "globally unique waypoint id",
      "customer_id": "customer id",
      "name": "waypoint name",
      "address": "physical address",
      "location": {
        "latitude": 3.0,
        "longitude": 4.0
      }
    },
  },

```

```
    "service_time_minutes": 999,
    "time_windows": [],
    "deliveries": [{
      "id": "globally unique delivery id",
      "product": "product sku or description",
      "quantity": 342.0
    }]
  },
  "vehicle": {
    "id": "globally unique vehicle id"
  }
}
```

}

Driver

Property	Type	Required	Description
id	string	true	The dc unique id of the driver.
name	string	false	The name of the driver.
phone	string	false	The phone number to contact the driver.

```
{
  "id": "globally unique driver id",
  "name": "driver name",
  "phone": "driver's phone"
}
```

Warehouse

 Note

Only the `id` property is required when importing a new route if and only if a warehouse with that same `id` has previously been imported into Foxtrot. If this is a new warehouse, all fields are required!

Property	Type	Required	Description
id	string	true	The id of the warehouse.
name	string	false	The name of the warehouse.
address	string	false	Physical address of the warehouse
location	Location	false	The location of the warehouse

```
{
  "id": "globally unique warehouse id",
  "name": "warehouse name",
  "address": "physical address",
  "location": {
    "latitude": 123.0,
    "longitude": 456.0
  }
}
```

Waypoint

! Important

If a waypoint id is supplied, it must be globally unique. You cannot have multiple waypoints share an id across routes or in the same route.

Property	Type	Required	Description
id	string	false	The globally unique id of the waypoint. If no
customer_id	string	true	The id of the customer at this waypoint.

Property	Type	Required	Description
name	string	false	The name of the waypoint.
address	string	false	Physical address of the waypoint
contact_name	string	false	The name of a contact for the waypoint
contact_phone	string	false	The phone number of a contact for the way
contact_email	string	false	The email address of a contact for the wayp
location	Location	false	The location of the waypoint
service_time_minutes	integer	false	The approximate service time needed to ser
time_windows	Array[TimeWindow]	false	An array of time windows for time restrictic
deliveries	Array[Delivery]	true	An array of deliveries at this stop. There mus

```
{
  "id": "globally unique waypoint id",
  "customer_id": "customer id",
  "name": "customer name",
  "address": "physical address",
  "location": {
    "latitude": 1.0,
    "longitude": 2.0
  },
  "service_time_minutes": 999,
  "time_windows": [{
    "start": 1510864915000,
    "end": 1510868515000
  }],
  "deliveries": [{
    "id": "delivery A",
    "product": "product",
    "quantity": 4355.0
  }]
}
```

TimeWindow

! Note

Time windows interval cannot be too restrictive. That threshold is configured per DC.

Property	Type	Required	Description
start	long	true	The start time of the time windows as a UNIX epoch since January 1st,
end	long	true	The end time of the time windows as a UNIX epoch since January 1st, :



```
{
  "start": 1510872073000,
  "end": 1510893673000,
}
```

Delivery

! Important

If a delivery id is supplied, it must be globally unique. You cannot have multiple deliveries share an id across routes or in the same route.

Property	Type	Required	Description
id	string	false	The globally unique id of this delivery. If not specified, a random id will l
product	string	true	The name of the product.
quantity	float	false	The quantity.



```
{  
  "id": "globally unique delivery id",  
  "product": "cases of beer",  
  "quantity": 3.0  
}
```

Vehicle

Property	Type	Required	Description
id	string	true	The globally unique id of the vehicle.

```
{  
  "id": "globally unique vehicle id"  
}
```

Location

Property	Type	Required	Description
latitude	float	true	The latitude coordinate point.
longitude	float	true	The longitude coordinate point.

```
{  
  "latitude": 12.0,  
  "longitude": 45.0  
}
```

Importing Invoiced Routes

You can import invoiced routes by using the [import invoiced routes](#) endpoint. The endpoint creates a route with invoices at each stop. After importing an invoiced route, you will need to check for the import status. See [example](#)

Models

Here are the models needed to import an invoiced route.

Invoiced Route

ⓘ Important

In order for a driver to receive a route, it must be [activated](#) after a successful import or you can set the *activate_after_successful_import* flag to true during route creation.

ⓘ Note

You can control the start and end location of a route by specifying a *warehouse*. If you want to define different start and end locations for a route, use the *start_warehouse* and *end_warehouse* parameters. If a *warehouse* or *start_warehouse* and *end_warehouse* are not supplied, Foxtrot will use the default warehouse location for that DC (Under Account > Settings > DC > Warehouse).

ⓘ Important

Route id has to be globally unique. You cannot have multiple routes with the same id.

❗ Important

If you are not sure the driver has been imported into Foxtrot, pass in the *driver* parameter and the driver will be imported if the driver does not exist. *driver_id* and *driver* cannot be used together.

Property	Type	Required	Description
id	string	true	The globally unique id of the route.
name	string	false	The name of the route, can be empty.
driver_id	string	false	The id of the driver to be assigned to this route.
driver	Driver	false	The driver assigned to this route.
activate_after_successful_import	boolean	false	Setting to automatically activate the route after a successful import.
start_time	long	true	The start time of the route.
warehouse	Warehouse	false	The route start and end location.
start_warehouse	Warehouse	false	The route starting location.
end_warehouse	Warehouse	false	The route ending location.
waypoints	Array[Invoiced Waypoint]	true	An array of waypoint objects.
vehicle	Vehicle	false	The vehicle to be associated with the route.

```
{
  "id": "globally unique route id",
  "name": "route name",
  "driver_id": "globally unique driver id",
  "activate_after_successful_import": false,
  "start_time": 1510860915000,
  "start_warehouse": {
    "id": "globally unique warehouse id",
    "name": "warehouse name",
    "address": "physical address",
    "location": {
      "latitude": 3.0,
      "longitude": 4.0
    }
  },
  "end_warehouse": {
    "id": "globally unique warehouse id",
    "name": "warehouse name",
    "address": "physical address",
    "location": {
      "latitude": 3.0,
      "longitude": 4.0
    }
  },
  "waypoints": [{
    "id": "globally unique waypoint id",
    "customer_id": "customer id",
    "name": "waypoint name",
    "address": "physical address",
    "location": {
      "latitude": 1.0,
      "longitude": 2.0
    }
  },
  "service_time_minutes": 999,
  "time_windows": [{
    "start": 1510864915000,
    "end": 1510868515000
  }],
  "invoices": [{
    "id": "globally unique invoice id",
    "payment_method": "CASH",
    "invoice_name_attributes": [{"key": "key1", "value": "value2"}],
    "items": [{
      "id": "globally unique item id",
      "sku": "sku A",
      "name": "name A",
      "quantity": 123.0,
      "display_unit_of_measure": "bottles",
      "total_price_cents": 9999,
      "currency": "USD"
    }]
  }],
},
```

```

{
  "id": "globally unique waypoint id",
  "customer_id": "customer id",
  "name": "waypoint name",
  "address": "physical address",
  "location": {
    "latitude": 3.0,
    "longitude": 4.0
  },
  "service_time_minutes": 999,
  "time_windows": [],
  "invoices": [{
    "id": "globally unique invoice id",
    "payment_method": "PREPAID",
    "invoice_name_attributes": [{"key": "key1", "value": "value2"}],
    "items": [{
      "id": "globally unique item id",
      "sku": "sku B",
      "name": "name B",
      "quantity": 123.0,
      "display_unit_of_measure": "bottles",
      "total_price_cents": 9999,
      "currency": "USD"
    }]
  }],
  "vehicle": {
    "id": "globally unique vehicle id"
  }
}

```

Driver

Property	Type	Required	Description
id	string	true	The dc unique id of the driver.
name	string	false	The name of the driver.
phone	string	false	The phone number to contact the driver.

```
{
  "id": "globally unique driver id",
  "name": "driver name",
  "phone": "driver's phone"
}
```

Invoiced Waypoint

❗ Important

The waypoint id, is later used to submit attempts via our SDK. If a waypoint id is supplied, it must be globally unique. You cannot have multiple waypoints share an id across routes or in the same route. If you don't uniquely identify stops, a good technique might be to concatenate your unique route id with the customer id.

Property	Type	Required	Description
id	string	false	The globally unique id of the waypoint. If no
customer_id	string	true	The id of the customer at this waypoint.
name	string	false	The name of the waypoint.
address	string	false	Physical address of the waypoint
location	Location	false	The location of the waypoint
service_time_minutes	integer	false	The approximate service time needed to ser
time_windows	Array[TimeWindow]	false	An array of time windows for time restrictic
invoices	Array[Invoice]	true	An array of invoices at this stop. There must

```

{
  "id": "globally unique waypoint id",
  "customer_id": "customer id",
  "name": "customer name",
  "address": "physical address",
  "location": {
    "latitude": 1.0,
    "longitude": 2.0
  },
  "service_time_minutes": 999,
  "time_windows": [{
    "start": 1510864915000,
    "end": 1510868515000
  }],
  "invoices": [{
    "id": "globally unique invoice id",
    "payment_method": "CASH",
    "invoice_name_attributes": [{"key": "key1", "value": "value2"}],
    "items": [{
      "id": "globally unique item id",
      "sku": "sku A",
      "name": "name A",
      "quantity": 123.0,
      "total_price_cents": 9999,
      "currency": "USD"
    }]
  }]
}

```

Invoice

⚠ Important

If an invoice id is supplied, it must be globally unique. You cannot have multiple invoices share an id across routes or in the same route.

Property	Type	Required	Description
id	string	false	The globally unique id of this in
payment_method	Payment Method	true	The payment method for this ir
invoice_name_attributes	Array[Invoice Name Attribute]	false	Extra attributes for this invoice

Property	Type	Required	Description
items	Array[Invoice Item]	true	An array of invoice items for th

```
{
  "id": "globally unique invoice id",
  "payment_method": "CASH",
  "invoice_name_attributes": [{"key": "Customer TAX ID", "value": "A-1232-001"}],
  "items": [{
    "id": "globally unique item id",
    "sku": "sku A",
    "name": "name A",
    "quantity": 123.0,
    "display_unit_of_measure": "bottles",
    "total_price_cents": 9999,
    "currency": "USD"
  }]
}
```

Payment Method

An enumeration of possible payment methods.

Method	Description
<i>CASH</i>	Payment is onsite with cash.
<i>CREDIT_CARD</i>	Payment is onsite with credit card.
<i>DEBIT_CARD</i>	Payment is onsite with debit card.
<i>BILLED</i>	Customer billed and payment will be received in the future.
<i>PREPAID</i>	Payment has already been received.

Invoice Name Attribute

Key-value object for additional information you would like to add to the invoice.

Property	Type	Required	Description
key	string	true	The name of the attribute.
value	string	true	The value of the attribute.

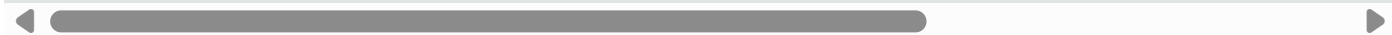
```
{
  "key": "Customer TAX ID",
  "value": "A-1232-001"
}
```

Invoice Item

ⓘ Important

If an invoice item id is supplied, it must be globally unique. You cannot have multiple invoice items share an id across routes or in the same route.

Property	Type	Required	Description
id	string	false	The globally unique id of this invoice item. If not specified, a unique id will be generated.
sku	string	true	The SKU of this product.
name	string	true	The name of this product.
quantity	float	true	The quantity.
display_unit_of_measure	string	false	The units to display next to quantity (eg. bottles, cases)
total_price_cents	integer	true	The total price for this line item including quantities, discounts, and taxes.
currency	string	false	The currency code representing the currency used to calculate the total price.



```
{  
  "id": "globally unique item id",  
  "sku": "sku A",  
  "name": "name A",  
  "quantity": 123.0,  
  "display_unit_of_measure": "bottles",  
  "total_price_cents": 9999,  
  "currency": "USD"  
}
```

```

response = POST_route(dc_id, route)
if is_http_status_202(response):
    txid = response.json()["data"]["txid"]

    while True:
        sleep(1)
        import_status_response = GET_route_import_status(dc_id, txid)
        if is_http_status_202(import_status_response):
            continue
        elif is_http_status_200(import_status_response):
            # Successfully imported the route
            route_id = import_status_response.json()["data"]["route_id"]
            break
        else:
            # Do error handling here.
    else:
        # Do error handling here.

```

Models

The models used by the `import route` endpoint are drastically different from the ones described in [models documentation](#). Here are the models needed by the import route endpoint for a successful import.

Importable Route

Property	Type	Required	Description
route	Route	true	The route information that is being imported

Route

Note

You can control the start and end location of a route by specifying a *warehouse*. If you want to define different start and end locations for a route, use the *start_warehouse* and *end_warehouse* parameters. If a *warehouse* or *start_warehouse* and *end_warehouse* are not

Invoice Status

After importing invoiced routes, invoices statuses can be checked using the [invoice status](#) endpoint.

Models

Invoice Status

Property	Type	Nullable	Description
id	string	false	The globally unique id of this inv
payment_method	Payment Method	false	The payment method for this in'
invoice_name_attributes	Array[Invoice Name Attribute]	false	Extra attributes for this invoice t
items	Array[Invoice Item Status]	false	An array of invoice item status.



Route Accounting

Additional financial information about the route may be tracked during the route to help with operational goals. Please use our [route accounting API](#) for other tracking options.

Models

Route Accounting Record 🔗

A Route Accounting record is used to audit how much cash is stored in different containers.

ⓘ Important

Route Accounting Record id has to be globally unique. You cannot have multiple records with the same id.

Property	Type	Nullable	Description
id	string	false	The globally unique id of the record.
cash_received_cents	float	false	The amount of cash in cents in the container.
cash_container_id	string	false	The id of the container containing the cash.
driver_id	string	false	Driver's id that placed the cash in the container.

```
{  
  "id": "globally unique record id",  
  "driver_id": "globally unique driver id",  
  "cash_received_cents": 9999,  
  "cash_container_id": "bag ABC"  
}
```

Webhooks

Foxtrot uses Webhooks to notify your service when certain events occur within Foxtrot.

Each Webhook consists of a URL, a list of events, and a list of DCs. Whenever an event occurs in one of the DCs, a message will be posted to the Webhook URL.

You can create, read, update, and delete Webhooks using the [Webhooks API methods](#).

Webhook Delivery

Webhooks notifications will be delivered to your Webhook URL as an HTTP *POST* in JSON format (with the header Content-Type: 'application/json' set):

```
{
  "event": "ROUTE_CREATED", // The event that triggered the webhook
  "hook_timestamp_millis": 1572557276345, // When the webhook event occurred
  "dc_id": "1" // Which DC this event occurred in
  "route_id": "2" // Which Route this event occurred in
}
```

The exact fields included in the body will vary depending on the type of event. See [WebhookEventType](#) for a full description of which data will be sent with which event.

Your Webhook processor should respond to the Webhook event with a *200 OK* response. If you do not respond with *200 OK*, Foxtrot will attempt to redeliver the Webhook event at a later time. However, if your service does not respond with a *200 OK* response within several hours, Foxtrot

will eventually drop that Webhook event.

If your Webhook continuously fails to respond to events for more than a day, Foxtrot may disable that Webhook and it'll have to be enabled manually.

Models

Webhook

```
{
  "id": "the-webhook-identifier",
  "webhook_url": "https://example.com/webhook",
  "enabled": true,
  "dc_ids": ["1", "2", "3"],
  "events": ["ROUTE_CREATED", "ROUTE_STARTED"]
}
```

Property	Type	Nullable	Description
id	string	false	The id of the Webhook.
url	string	false	The name of the route.
dc_ids	Array[string]	false	An array of DC ids for which this Webhook will rec
events	Array[WebhookEventType]	false	An array of Event Types for which this Webhook v

WebhookEventType

An enumeration of possible webhook event types.

Method	Description
<i>ROUTE_CREATED</i>	Triggered when a route is created.
<i>ROUTE_STARTED</i>	Triggered when a driver clicks on Route Start or the start route API is
<i>ROUTE_ETAS_CHANGED</i>	Triggered when a route's ETAs changed, after an optimization or ETA
<i>WAYPOINT_STATUS_CHANGED</i>	Triggered when a waypoint's delivery gets a delivery attempt.
<i>CUSTOMER_LOCATION_CHANGED</i>	Triggered when a customer's location changes.